

Estado, función y programa del agente

Preguntas y respuestas para repasar los temas trabajados en clase: qué es el estado de un agente, la diferencia entre función y programa, qué significa “implementar” y cómo se relacionan las percepciones con el objetivo.

- Bloque 1: Estado ● Bloque 2: Función y programa ● Bloque 3: Implementación

- Bloque 4: Percepciones y objetivo

Cómo usar este material. Lee cada pregunta y, antes de mirar la respuesta, intentá contestarla con tus palabras. Al final hay un resumen y preguntas de práctica con su clave de respuestas.

1. ¿Qué significa “estado” cuando hablamos de agentes inteligentes?
2. ¿Qué es el estado del entorno? Dé un ejemplo.
3. ¿Qué es el estado interno y por qué hace falta?
4. ¿Cómo se actualiza el estado interno?
5. ¿Qué son los estados futuros y qué agentes los usan?
6. El agente reactivo simple, ¿tiene estado?
7. ¿Cómo usa el estado cada tipo de agente?
8. ¿Qué es la función del agente?
9. ¿Qué es el programa del agente?
10. ¿Por qué la función se define sobre la secuencia de percepciones y no solo sobre la última?
11. Si un agente siempre reacciona igual (por ejemplo, una caldera que prende debajo de 20 °C y apaga arriba de 23 °C), ¿no alcanza con una tabla de pocas entradas?
12. ¿Qué otros ejemplos muestran que la misma percepción puede exigir acciones distintas?
13. Entonces, ¿hay que construir esa tabla infinita?
14. Si el programa implementa la función, ¿qué pasa ante percepciones nuevas que no están escritas en el programa?
15. ¿Qué significa exactamente “el programa implementa la función”?
16. ¿Todo programa define una función?
17. ¿Y si aparece algo que el diseñador no previó?
18. Si quiero que la caldera también reaccione a la humedad, ¿hay que escribir una nueva función y modificar el programa?
19. ¿Las percepciones se eligen según el objetivo del agente?
20. El diseñador elige los sensores; ¿entonces también elige qué valores llegan?
21. Si cambian las percepciones, ¿siempre hay que reescribir el programa desde cero?

El estado de un agente

1 ¿Qué significa “estado” cuando hablamos de agentes inteligentes?

Es la **descripción de cómo está el mundo en un momento dado**: una “foto” de la situación. No es un estado de ánimo ni si el agente está encendido o apagado.

En la materia aparece con **tres sentidos** relacionados: el estado del entorno (cómo es el mundo de verdad), el estado interno (cómo cree el agente que es el mundo) y los estados futuros (cómo sería el mundo si hiciera tal acción).

2 ¿Qué es el estado del entorno? Dé un ejemplo.

Es la situación **real** del mundo, con todos sus detalles relevantes. En el mundo de la aspiradora un estado se describe con tres datos: dónde está la aspiradora, si A está sucio y si B está sucio. Ejemplo: (*aspiradora en A, A sucio, B limpio*).

Hay **8 estados posibles** ($2 \text{ ubicaciones} \times 2 \times 2$). Las acciones hacen que el mundo pase de un estado a otro: si aspira, pasa de (*A, A sucio, B limpio*) a (*A, A limpio, B limpio*).

3 ¿Qué es el estado interno y por qué hace falta?

Es la **mejor suposición del agente sobre cómo está el mundo**, incluyendo lo que no ve en este momento. Hace falta porque el agente no siempre percibe el estado completo: la aspiradora solo ve su cuadrado. El mundo tiene 8 estados, pero ella solo puede distinguir 4 percepciones.

Es una **creencia, no una certeza**: puede equivocarse. Lo usan los agentes basados en modelos.

Ejemplos: la aspiradora con memoria percibe [*A, Limpio*] y recuerda que limpió B, así que no se mueve · el taxi ya no ve el auto en su punto ciego, pero recuerda que está ahí y no cambia de carril · vos, con un corte de luz, recordás dónde está la mesa y la esquivás.

4 ¿Cómo se actualiza el estado interno?

En cada ciclo, con la función **ACTUALIZAR-ESTADO**, que combina:

Ingrediente	Qué aporta	Ejemplo (aspiradora con memoria)
Estado anterior	Lo que el agente creía antes	Creía: “B estaba sucio”
Última acción	Lo que acaba de hacer	Acaba de aspirar en B
Percepción nueva	Lo que ve ahora	Percibe [B, Limpio]
Modelo de transición	Cómo cambia el mundo, solo y por sus acciones	“Aspirar deja limpio el cuadrado actual”
Modelo sensorial	Cómo se refleja el mundo en los sensores	“Si el sensor dice Limpio, está limpio”
Resultado	Nuevo estado interno	“A limpio (lo recuerdo) y B limpio (lo veo)” → no hacer nada

5 ¿Qué son los estados futuros y qué agentes los usan?

Son estados que el agente **imagina con su modelo** para evaluar una acción antes de hacerla: “¿cómo será el mundo si hago la acción A?”.

- El agente **basado en objetivos** pregunta si ese estado futuro cumple su meta (sí / no).
- El agente **basado en utilidad** le asigna un número: qué tan bueno es ese estado.

6 El agente reactivo simple, ¿tiene estado?

En su programa aparece una variable llamada `estado` (`estado ← INTERPRETAR-ENTRADA(percepción)`), pero es solo una descripción abstracta de **lo que ve ahora mismo**, sin memoria. Por eso falla cuando el entorno es parcialmente observable: su “estado” nunca incluye lo que no está viendo.

7 ¿Cómo usa el estado cada tipo de agente?

Tipo de agente	Qué hace con el estado	¿Memoria?	¿Imagina el futuro?	¿Valora con un número?
Reactivo simple	Lo arma solo con la percepción actual	No	No	No
Basado en modelos	Lo guarda y lo actualiza	Sí	No	No
Basado en objetivos	Predice estados futuros y los compara con la meta	Sí	Sí	No (cumple / no cumple)
Basado en utilidad	Le da un valor a cada estado futuro y elige el mejor	Sí	Sí	Sí
Que aprende	Usa cualquiera de los anteriores y mejora sus modelos	Sí	Según su base	Según su base

Función del agente y programa del agente

8 ¿Qué es la función del agente?

Imaginá que no podés abrir al agente: solo observás desde afuera qué le llega por los sensores y qué hace. Si anotarás su respuesta ante **cada situación posible**, tendrías una descripción completa de su comportamiento. Eso es la **función del agente**.

Es una **descripción matemática y abstracta**: dice qué hace, no cómo está construido. Formalmente: **f : secuencia de percepciones $\rightarrow$ acción**. Se la puede imaginar como una tabla, casi siempre infinita.

9 ¿Qué es el programa del agente?

Es la **implementación concreta**: código que corre en una máquina real, recibe la **percepción actual** y devuelve una acción. Dice cómo se logra el comportamiento. Es la “mente”; sensores y actuadores son el “cuerpo”.

Agente = Arquitectura + Programa

Analogías: $f(x) = x^2$ es la función; `return x*x` es un programa que la implementa • el reglamento dice qué cobrar ante cada jugada (función); el árbitro lo aplica en la cancha (programa).

10 ¿Por qué la función se define sobre la secuencia de percepciones y no solo sobre la última?

Porque tiene que poder describir **a cualquier agente**, incluidos los que recuerdan. Quien observa desde afuera no sabe si el agente tiene memoria, así que la descripción más general es: “dado todo lo que percibió hasta ahora, ¿qué hace?”.

Definirla sobre secuencias **no obliga** a usar la historia: solo **permite** describir agentes que la usan.

11 Si un agente siempre reacciona igual (por ejemplo, una caldera que prende debajo de 20 °C y apaga arriba de 23 °C), ¿no alcanza con una tabla de pocas entradas?

En parte sí. Para un agente que no usa la historia, la tabla por secuencias es redundante: todas las filas que terminan en la misma percepción tienen la misma acción. Es el caso del

agente reactivo simple, un caso particular de la función general (como una $f(x, y)$ que ignora la y).

Secuencia	Acción
[19]	Prender
[25, 19]	Prender
[22, 21, 19]	Prender

Pero la caldera del ejemplo sí depende de la historia. Si el sensor marca $21,5^{\circ}\text{C}$, ¿debe estar prendida o apagada? Con la percepción actual sola no se puede saber:

Secuencia de percepciones	Qué está pasando	Acción correcta
..., 19, 20,5, 21,5	Viene calentando desde abajo	Seguir prendida hasta 23
..., 24, 22,5, 21,5	Viene enfriando desde arriba	Seguir apagada hasta bajar de 20

Misma percepción ($21,5^{\circ}\text{C}$), acciones distintas. La diferencia está en la historia.

La franja entre 20 y 23 existe a propósito, para que la caldera no se prenda y apague cada pocos segundos. Si el termostato también **percibiera** si la caldera está prendida, ese dato sería parte de la percepción actual y podría ser reactivo simple. Si solo percibe la temperatura, necesita memoria.

12 ¿Qué otros ejemplos muestran que la misma percepción puede exigir acciones distintas?

- **Máquina expendedora:** cada percepción es igual (“entró una moneda de \$100”); con la primera y la segunda espera, con la tercera entrega el producto.
- **Cerradura con clave:** el primer y el segundo “clave incorrecta” dan “intente de nuevo”; el tercero bloquea la cuenta.
- **Chatbot:** la percepción “sí” significa comprar si antes preguntó “¿confirmás la compra?”, y cancelar si preguntó “¿querés cancelar?”.
- **Aspiradora sin sensor de ubicación:** percibe “Limpio” y no sabe si ir a la derecha o a la izquierda, salvo que recuerde de dónde vino.

13 Entonces, ¿hay que construir esa tabla infinita?

No. La función es la descripción ideal (puede ser infinita); el **programa** es la forma **compacta** de lograr el mismo comportamiento. La caldera no necesita guardar toda la historia: le alcanza con **un dato que la resume**.

```

función TERMOSTATO(temperatura) devuelve acción
  persistente: encendida // un solo dato que resume toda la historia
  si temperatura < 20 entonces encendida ← verdadero
  si temperatura > 23 entonces encendida ← falso
  si encendida entonces devolver Prender
  si no devolver Apagar

```

La variable `encendida` es un **estado interno**: resume la historia sin guardarla entera.

	Función del agente	Programa del agente
Qué es	Descripción de qué hace ante cada historia posible	Código concreto que lo logra
Recibe	Toda la secuencia de percepciones	Solo la percepción actual (y lo que guardó en memoria)
Tamaño	Puede ser infinita	Debe ser pequeño
Para la caldera	“Ante...19, 21,5 → prender; ante...24, 21,5 → apagar;...”	<code>encendida</code> + tres reglas

El gran desafío de la IA: pasar de la función gigante a un programa pequeño.

Qué significa que el programa implementa la función

14 Si el programa implementa la función, ¿qué pasa ante percepciones nuevas que no están escritas en el programa?

El programa **no enumera casos: calcula la respuesta**. Una regla cubre infinitos casos.

`return x*x` no tiene guardado ningún resultado, pero sirve para cualquier número, incluso 7.348.201, que nadie le “enseñó”.

Con la caldera pasa lo mismo: en ningún lado dice “21,37 °C”, pero la regla “< 20” cubre cualquier valor menor que 20.

Percepción	¿Estaba escrita?	Qué calcula el programa	Acción
18	No	$18 < 20 \rightarrow$ encendida = verdadero	Prender
21,37	No	No cumple ninguna regla $\rightarrow$ queda verdadero	Prender
23,4	No	$23,4 > 23 \rightarrow$ encendida = falso	Apagar
21,37	No	No cumple ninguna regla $\rightarrow$ queda falso	Apagar

Ninguna temperatura estaba en el programa y todas tienen respuesta. Además, 21,37 aparece dos veces con acciones distintas: la historia resuelta con una sola variable.

15 ¿Qué significa exactamente “el programa implementa la función”?

Que si le das al programa **cualquier** secuencia de percepciones posible, una por una, la acción que devuelve al final es la que la función indica para esa secuencia. No hace falta que el programa “conozca” la secuencia de antemano: alcanza con que sus reglas la cubran.

16 ¿Todo programa define una función?

Sí. Aunque nadie haya escrito la tabla, si probaras el programa con cada secuencia posible y anotaras qué hace, obtendrías su tabla completa. La tabla es la descripción desde afuera; el programa es el mecanismo de adentro que la genera.

17 ¿Y si aparece algo que el diseñador no previó?

Hay que distinguir dos situaciones:

1. Una percepción que los sensores no pueden producir. El termostato no percibe la humedad ni el color de la pared: no tiene sensor para eso. La función se define solo sobre las secuencias que **sus sensores pueden generar**.

2. Un caso posible que el diseñador pensó mal. Si el programa solo dice “< 20 prender” y “> 23 apagar”, con 21 °C no entra en ninguna regla. Sigue definiendo **una** función, pero **no la que queríamos**: es un error de diseño.

Hay una función ideal (la de un agente racional) y la función que realmente produce nuestro programa. Diseñar bien es que coincidan lo más posible.

Cuando el entorno es demasiado complejo para anticipar todo, la solución es un **agente que aprende**: ajusta su programa con la experiencia y así cambia la función que implementa.

Percepciones, sensores y objetivo

- 18** Si quiero que la caldera también reaccione a la humedad, ¿hay que escribir una nueva función y modificar el programa?

Sí. Pasan tres cosas, en este orden:

1. **Cambia la arquitectura:** hay que agregar un sensor de humedad. Sin sensor, la humedad no existe para el agente.
2. **Cambia la función:** la percepción pasa de [21,5] a [21,5 °C; 75 %]. Las secuencias posibles son otras: es una función nueva.
3. **Cambia el programa:** tiene que leer el dato nuevo y usarlo.

```
función TERMOSTATO([temperatura, humedad]) devuelve acción
  persistente: encendida
  umbral ← 20
  si humedad > 70 entonces umbral ← 21 // con humedad alta se siente más frío
  si temperatura < umbral entonces encendida ← verdadero
  si temperatura > 23 entonces encendida ← falso
  si encendida entonces devolver Prender
  si no devolver Apagar
```

Agente = Arquitectura + Programa: si tocás los sensores, casi siempre tenés que tocar también el programa.

- 19** ¿Las percepciones se eligen según el objetivo del agente?

Sí. El cambio no empieza por el sensor, sino por el objetivo. Si la medida de rendimiento pasa de “mantener la temperatura” a “que las personas estén cómodas”, la humedad empieza a importar. El orden natural del diseño sigue el esquema REAS:

Rendimiento → Entorno → Actuadores y Sensores → Función → Programa

El diseñador pone los sensores que dan la información necesaria para cumplir la medida de rendimiento, dentro de lo posible y razonable en costo.

- 20** El diseñador elige los sensores; ¿entonces también elige qué valores llegan?

No. Elegís poner un termómetro, pero no que marque solo “valores lindos”. Si se rompe y marca -40 °C, o una ventana abierta hace bajar la temperatura de golpe, eso también es una percepción posible. El programa tiene que responder razonablemente ante **todo lo que ese**

sensor puede producir. El taxi elige tener cámaras, pero no elige qué aparece delante de ellas.

21 Si cambian las percepciones, ¿siempre hay que reescribir el programa desde cero?

No necesariamente; depende de cómo esté hecho:

- Si es un **intérprete general de reglas** (como REGLA-COINCIDENCIA del reactivo simple), a veces alcanza con agregar reglas nuevas.
- Si es un **agente que aprende**, puede descubrir con la experiencia cuánto influye un dato (por ejemplo, la humedad) sin que nadie le escriba el umbral.

Pero agregar un **tipo nuevo** de percepción, de un sensor que antes no existía, sí requiere conectar ese dato al programa.

Resumen para memorizar

Estado

Descripción de cómo está el mundo. Del entorno (real), interno (lo que el agente cree) o futuro (lo que imagina si hace una acción).

Función del agente

Qué acción corresponde a cada secuencia de percepciones posible. Abstracta, a menudo infinita.

Programa del agente

Código concreto y compacto que, con reglas generales y memoria, produce el comportamiento de la función.

Percepciones y objetivo

El objetivo define qué percibir → sensores → percepciones posibles → función → programa.

Preguntas de práctica

1. Una persona dice: “la función del agente es el código que corre en la computadora”. ¿Qué está mal?
2. ¿Por qué un agente reactivo simple puede describirse igual con una función sobre secuencias?
3. Dé un ejemplo propio en el que la misma percepción requiera acciones distintas.
4. ¿Qué es un estado interno y en qué tipo de agente aparece por primera vez?
5. ¿Cuál es la diferencia entre el estado del entorno y el estado interno?
6. Un programa no contempla un caso posible y no devuelve acción. ¿Implementa alguna función?
7. Si a un robot de limpieza se le agrega una cámara, ¿qué cambia según la cadena REAS?

Clave de respuestas

1. Confunde función con programa. La función es la descripción abstracta (qué acción para cada secuencia de percepciones); el código es el programa, que la implementa.
2. Porque las secuencias que terminan en la misma percepción reciben la misma acción: la función existe sobre secuencias, pero en la práctica solo depende del último elemento.
3. Por ejemplo, un ascensor que percibe “botón 5 presionado”: si ya está yendo al 5 no hace nada nuevo; si está detenido, arranca. (Cualquier ejemplo donde el pasado cambie la acción es válido.)
4. Es la mejor suposición del agente sobre cómo está el mundo, incluyendo lo que no ve. Aparece en el agente reactivo basado en modelos.
5. El estado del entorno es cómo es el mundo de verdad; el estado interno es lo que el agente cree que es el mundo. Pueden no coincidir.
6. Sí, implementa una función, pero no la ideal que el diseñador quería. Es un error de diseño.
7. Cambian los sensores (arquitectura), por lo tanto las percepciones posibles, la función y el programa, que debe usar el dato nuevo.